Orderflow Engine — Technical manual

Jesús Ariel González Bonilla

2026-10-17

Question this document answers: how do I install, configure, run, operate, test and maintain Orderflow Engine?

About this manual

Audience

This manual is for the engineer who deploys, operates and maintains Orderflow Engine: the pricing engine and idempotent order confirmation of a multi-country B2B beverage marketplace, its Angular web client and the Docker Compose stack that runs them. It assumes working knowledge of Docker Compose, PostgreSQL, a Unix shell and HTTP. How a shopkeeper uses the client is in the user manual (user-manual.md), and why the system is shaped this way is in the architecture report (architecture-report.md).

Conventions

Convention	Meaning
bash blocks	Commands for bash on Linux, macOS or Git Bash on Windows. When the folder matters, a comment on the first line names it. Code and its comments are in English
text blocks	Output, SQL to type in psql, YAML or JSON
Paths	Relative to the workspace root, the folder that holds the orderflow-engine-* repositories side by side
<value>	A placeholder to replace, for example <correlationId>
Values	Every port, key and default shown is the default of .env.example. A .env can change it
Money	Amounts travel as JSON strings at the currency's scale: "71103" in COP, "77.11" in USD
Notices	**Note:** adds context. **Warning:** marks an action that deletes data or exposes the system
Sources	Each fact names the file it comes from, so it can be checked in the repositories

The system at a glance

Components and ports

Component	Built from	Host port	Network address	Role
db	postgres:17	Not published	db:5432	Single source of truth: catalog, levers, credit, quotes, orders, idempotency

				keys and outbox
api	orderflow-engine-api: Java 21, Spring Boot 4.1.1, run as a non-root user on eclipse-temurin:21-jre	8080 (API_PORT)	api:8080	Prices carts, seals quotes, confirms orders, serves order history, relays outbox events and purges expired data
web	orderflow-engine-web: Angular 22, served by nginxinc/nginx-unprivileged:1.30.6-alpine	8081 (WEB_PORT)	web:8080	The client. Its nginx forwards /api/ to the API
Push gateway and ERP	Fakes inside the API (adapters.out.push, adapters.out.erp)	—	—	Receive each confirmed order only through the outbox relay, and deduplicate by event_id

Source: orderflow-engine-infra/compose.yaml, the two Dockerfile files and orderflow-engine-docs/spec/architecture.md §1.

Request path

1. The browser loads the client from http://localhost:8081. The client calls only relative paths under /api/v1/.
2. nginx in the web container forwards /api/ unchanged to API_UPSTREAM, which is http://api:8080 in the full stack. The browser sees one origin, so the API needs no CORS. Request headers (X-Tenant-Id, X-Customer-Id, Idempotency-Key, Accept-Language, X-Correlation-Id) and response headers (Location, Idempotent-Replayed) pass through unchanged. Any other path falls back to index.html, so a reload on a client route works. /actuator is not forwarded.
3. The API validates headers and body, runs the use case and reads or writes PostgreSQL through JdbcClient repositories with explicit SQL and locks. A confirmation is one ACID transaction: idempotency key, quote lock, credit, order, two outbox events and the stored response. The 201 is sent only after the commit.
4. The outbox relay, a scheduled job inside the same API process, claims due events, calls the push and ERP fakes outside any database transaction, and marks each event SENT, schedules a retry or marks it DEAD.

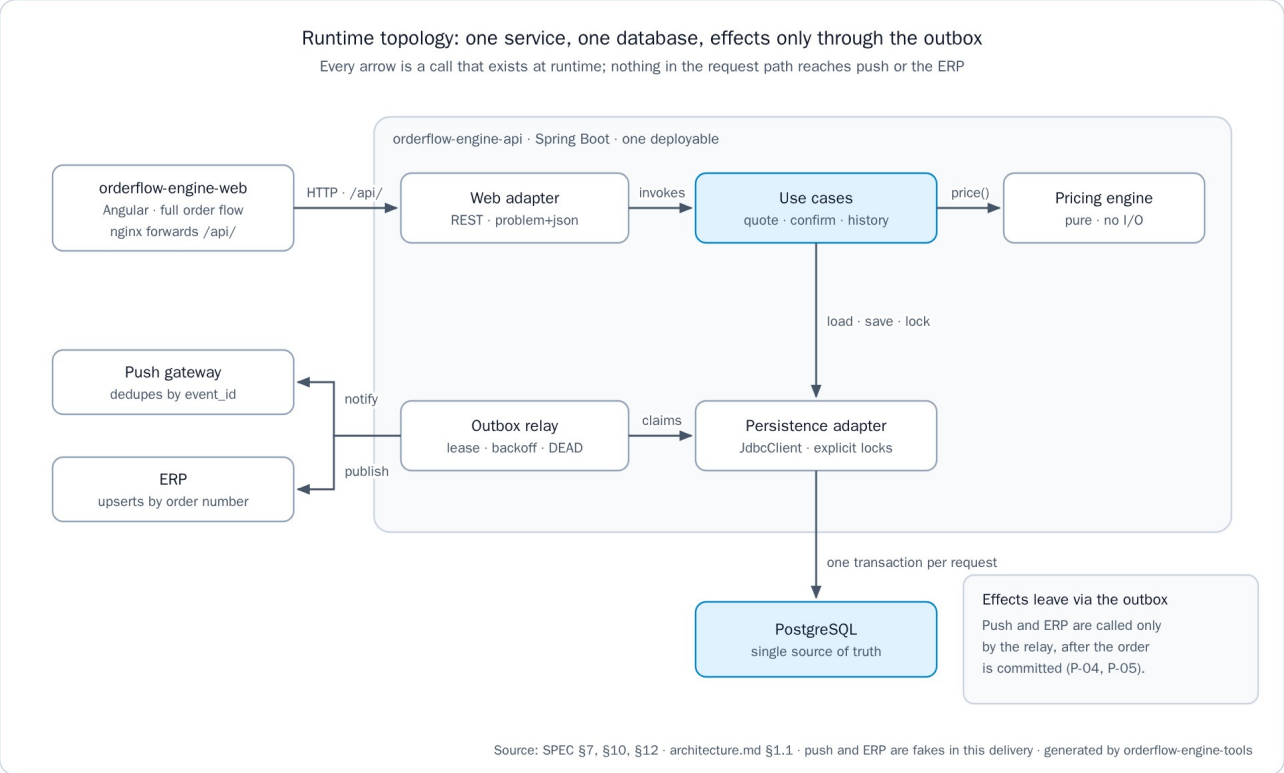


Figure 1. Runtime topology: the web client and its nginx, the API with its pricing engine and outbox relay, PostgreSQL, and the push and ERP fakes

Repositories

Repository	Content	Needed for
orderflow-engine-api	The deliverable: code, tests, README, ADR-0001 and the Liquibase changelogs	The stack and test-all
orderflow-engine-web	Angular client, nginx template, store list public/config/stores.json, product images and specs	The stack and test-all
orderflow-engine-infra	Full-stack compose.yaml, scripts/test-all.sh, scripts/test-all.ps1, scripts/e2e.sh, scripts/e2e.ps1 and the Playwright suite in e2e/	The stack (run Compose here) and test-all
orderflow-engine-docs	Specification, API contract, data model, architecture, diagrams, governance, handoff and delivery documents	test-all
orderflow-engine-tools	Documentation and integrity checks, figure and document generators	test-all
orderflow-engine-app	Reserved for a future mobile app, not built	test-all, whose checks read every repository

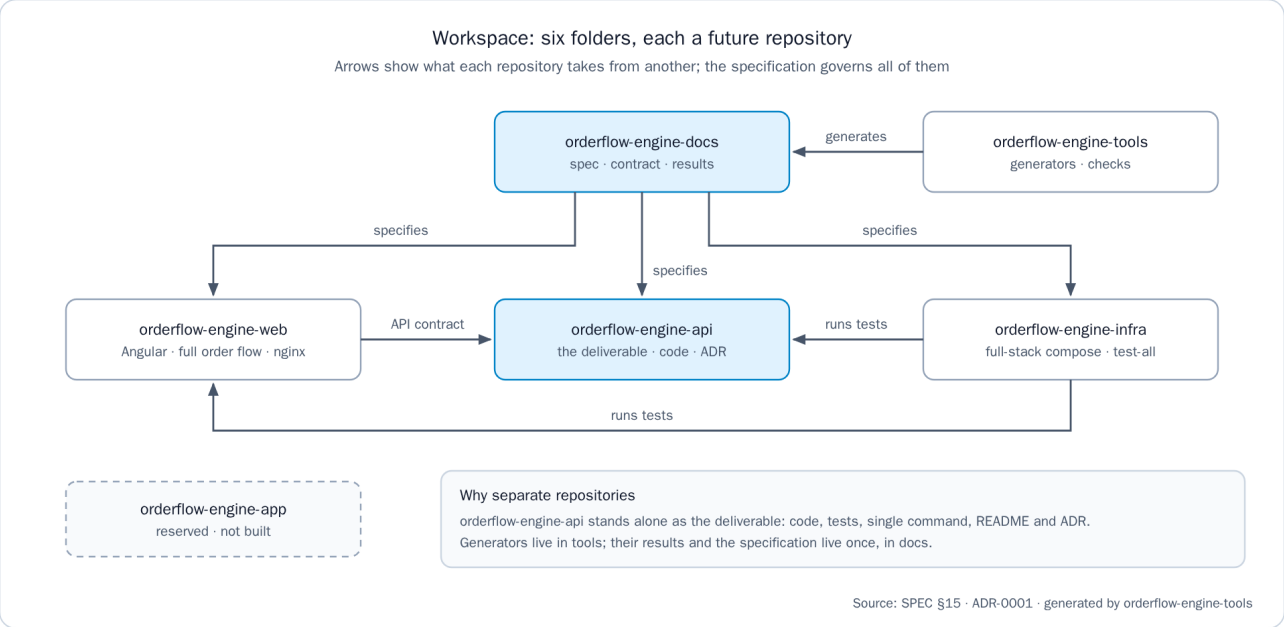


Figure 2. The six repositories of the workspace and what each one takes from another

Seeded data

Liquibase seeds four tenants, one per country of the assignment. Every country rounds HALF_UP, and a quote is held for 15 minutes. Both values are country data in the countries table.

Tenant	Country	Currency and scale	Tax rates	Gift tax	Levers
co-alpha	Colombia (CO)	COP, 0	19%, chips 5%	TAXED	Tier on BEV-COLA-350 (10+ 5%, 20+ 10%). Combo 2 BEV-COLA-350 + 1 SNK-CHIPS-45 for 5000. 1 BEV-WATER-600 per 10 snacks, repeatable
ec-alpha	Ecuador (EC)	USD, 2	15%	EXEMPT	Tier on the colas family (12+ 4%, 24+ 8%). Combo 6 BEV-COLA-350 + 2 SNK-PLANTAIN-45 for 3.99. 2 BEV-WATER-600 from a gross subtotal of 50.00, once
gt-alpha	Guatemala (GT)	GTQ, 2	12%	EXEMPT	Tier on BEV-COLA-350 (12+ 5%, 24+ 10%). 1 BEV-WATER-600 per 12 BEV-COLA-350,

					repeatable
pe-alpha	Peru (PE)	PEN, 2	18%	TAXED	Combo 2 BEV-COLA-350 + 1 SNK-CHIPS-45 for 6.00. 1 BEV-WATER-600 from a gross subtotal of 100.00, once

Store	Tenant	Customer id	Pays with
Tienda Norte	co-alpha	01a13cdb-cc00-7001-8001-000000000001	Credit up to 500,000, or cash
Tienda Sur	co-alpha	01a13cdb-cc00-7001-8001-000000000002	Cash only
Tienda Quito	ec-alpha	01a13cdb-cc00-7002-8001-000000000001	Credit up to 1,500.00, or cash
Tienda Guayaquil	ec-alpha	01a13cdb-cc00-7002-8001-000000000002	Cash only
Tienda Ciudad de Guatemala	gt-alpha	01a13cdb-cc00-7003-8001-000000000001	Credit up to 5,000.00, or cash
Tienda Quetzaltenango	gt-alpha	01a13cdb-cc00-7003-8001-000000000002	Cash only
Tienda Lima	pe-alpha	01a13cdb-cc00-7004-8001-000000000001	Credit up to 2,000.00, or cash
Tienda Arequipa	pe-alpha	01a13cdb-cc00-7004-8001-000000000002	Cash only

Source: the seed changelogs in orderflow-engine-api/src/main/resources/db/changelog/2026/ and orderflow-engine-web/public/config/stores.json.

Prerequisites

Need	Version	Used for
Docker with Compose v2 (docker compose)	A running daemon	The stack, the API integration tests (Testcontainers), the web tests and the end-to-end flow
Git	Any recent version	Cloning the repositories
JDK	21	./mvnw verify and test-all. The Maven Wrapper fetches Maven, so no global Maven is needed
Node	18 or later	Only the documentation and integrity checks of test-all and the version check of the end-to-end

		flow. The web tests and the end-to-end flow bring their own Node in their containers
Node, for the client on the host	22.22.3 or later in the 22 line, or 24.15 or later	Only npm test or npm start outside a container (Angular 22 requires it)
PowerShell	7 (pwsh)	Only scripts/test-all.ps1 on Windows
Python 3.11 and LibreOffice	With the packages of orderflow-engine-tools/requirements.txt	Only to regenerate the figures, the Word and PDF documents and the PowerPoint decks

Verified environment. The build machine recorded in orderflow-engine-docs/handoff/HANDOFF.md §5 has Java 21, Node 24 (24.20), Docker Desktop, Python 3.11 with python-docx, pypdf and python-pptx, and LibreOffice. On 2026-10-17 test-all ended with test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check) on fresh clones of every repository.

Installation and first start

Clone the repositories

Clone the six repositories side by side into an empty folder, which becomes the workspace root:

```
# From the folder that becomes the workspace root
for r in api web infra docs tools app; do
  git clone https://github.com/ariel5253/orderflow-engine-$r.git
done
```

```
workspace/
├── orderflow-engine-api/
├── orderflow-engine-web/
├── orderflow-engine-infra/   <- run docker compose here
├── orderflow-engine-docs/
├── orderflow-engine-tools/
└── orderflow-engine-app/
```

> **Note:** docker compose needs only orderflow-engine-api and orderflow-engine-web beside orderflow-engine-infra. test-all needs all six, because the documentation check reads every repository.

Configure the environment

orderflow-engine-infra/.env.example lists every key with a comment. Its values are the defaults written after :- in compose.yaml, so the stack starts without a .env. Create one only to change a value:

```
# From orderflow-engine-infra
cp .env.example .env
# Edit .env, for example DB_PASSWORD, API_PORT or WEB_PORT
```

> **Warning:** .env is never committed (it is listed in .gitignore). Set a real DB_PASSWORD before the first start: the PostgreSQL image stores the password when it creates the db-data volume, so a later change in .env does not reach an existing database.

Start the stack

```
# From orderflow-engine-infra
docker compose up --build -d
docker compose ps
```

--build builds the API image from ../orderflow-engine-api and the web image from ../orderflow-engine-web. The API image packages the service with tests skipped, because ./mvnw verify is the test command. The web image runs npm run build and never builds the test stage. Without -d, the logs of the three services stay in the terminal.

What happens at start

1. db starts PostgreSQL 17 on the db-data volume. Its health check runs pg_isready every 5 seconds, with 12 retries after a 10-second start period.
2. api starts only once db is healthy. Liquibase applies db/changelog/db.changelog-master.yaml: on an empty database, 33 changesets in 8 changelogs create the schema and seed the four tenants (log lines Running Changeset: ...). On later starts it applies only changesets it has not recorded in databasechangelog. Then the service listens on API_PORT (log line Started Application in ... seconds, about 10 seconds on the build machine).
3. web starts after api. The nginx image renders API_UPSTREAM into its configuration and serves the client on port 8080 of the container, published on WEB_PORT.
4. The outbox relay polls every API_OUTBOX_POLL_INTERVAL. The housekeeping job runs one hour after start, then every hour.

Check the API, then open the client:

```
curl http://localhost:8080/actuator/health
```

```
{"groups":["liveness","readiness"],"status":"UP"}
```

Open http://localhost:8081. The client starts in Spanish with the first store of stores.json (Tienda Norte, co-alpha). Add ?lang=en, or use the ES/EN switch, for English.

> **Note:** the API alone (docker compose up --build in orderflow-engine-api) and the client alone (the same command in orderflow-engine-web) publish the same ports as the full stack. Run one stack at a time, or give one of them another API_PORT or WEB_PORT.

Configuration

Full stack keys

Every key of orderflow-engine-infra/.env.example, the Spring property or container setting it feeds, and its effect:

Key	Default	Sets	Effect
DB_NAME	orderflow	POSTGRES_DB and the database of SPRING_DATASOURCE_URL (jdbc:postgresql://db:5432/<DB_NAME>)	Name of the database
DB_USER	orderflow	POSTGRES_USER and SPRING_DATASOURCE_USERNAME	Role the API connects as. Liquibase migrates with it too
DB_PASSWORD	change-me	POSTGRES_PASSWORD and SPRING_DATASOURCE_PASSWORD	Password of that role. Set a real value in .env
API_PORT	8080	SERVER_PORT (server.port), the published port, and API_UPSTREAM of web (http://api:<API_PORT>)	HTTP port of the API, inside the container and on the host
API_DEFAULT_LOCALE	es	APP_I18N_DEFAULT_LOCALE (app.i18n.default-locale)	Language of error texts when Accept-Language is missing or names an unsupported language
API_OUTBOX_POLL_INTERVAL	1s	app.outbox.poll-interval	Wait between two polls for due events
API_OUTBOX_LEASE	30s	app.outbox.lease	How long a claimed event stays leased. After it, another poll may claim and send it again
API_OUTBOX_MAX_ATTEMPTS	8	app.outbox.max-attempts	Delivery attempts before an event becomes DEAD
API_OUTBOX_BACKOFF_BASE	2s	app.outbox.backoff.base	First ceiling of the retry wait. The

			ceiling doubles after each failed attempt, and the wait is drawn at random below it
API_OUTBOX_BACKOFF_MAX	5m	app.outbox.backoff.max	Highest ceiling of the retry wait
API_IDEMPOTENCY_RETENTION	24h	app.idempotency.retention	How long an idempotency key replays its first response
API_HOUSEKEEPING_RETENTION	7d	app.housekeeping.retention	How long expired unconfirmed quotes and SENT outbox events are kept
WEB_PORT	8081	The published port of web, mapped to port 8080 of the container	Host port of the client

Durations use Spring's notation (1s, 30s, 5m, 24h, 7d). After a change in .env, run docker compose up -d: Compose recreates the services whose configuration changed.

Some values are not configuration on purpose:

Value	Where it lives
lock_timeout of a confirmation: 2 seconds	ConfirmOrderUseCase.LOCK_TIMEOUT in the code
Housekeeping schedule: every hour	HousekeepingJob in the code
Currency scale, rounding mode, gift tax policy and quote time to live	Country data in the currencies and countries tables, changed with a changeset
Datasource URL, user and password	Only the environment (SPRING_DATASOURCE_*). application.yml sets no default

Keys of the standalone stacks and the end-to-end script

Where	Key	Default	Effect
orderflow-engine-api/.env.example	The same DB_* and API_* keys	The same defaults	The API and its PostgreSQL alone, started from orderflow-engine-api
orderflow-engine-web/.env.example	WEB_PORT	8081	Host port of the client alone
orderflow-engine-web/.env.example	API_UPSTREAM	http://host.docker.internal:8080	Where nginx forwards /api/, read when the container starts: a new value

			needs a restart, not a rebuild. extra_hosts maps host.docker.internal to the host gateway on Linux
Environment of scripts/e2e.sh and scripts/e2e.ps1	E2E_PROJECT	ofe-e2e	Compose project name of the end-to-end stack
Environment of scripts/e2e.sh and scripts/e2e.ps1	E2E_API_PORT	18080	Host port of its API
Environment of scripts/e2e.sh and scripts/e2e.ps1	E2E_WEB_PORT	18081	Host port of its client
Environment of npm start (proxy.conf.mjs)	API_UPSTREAM	http://localhost:8080	Where the Angular development server forwards /api/

Operation

Start, stop and restart

```
# From orderflow-engine-infra
docker compose up --build -d     # build and start, or apply new code or a changed .env
docker compose ps                # state of db (with its health), api and web
docker compose stop              # stop the three services; the data stays
docker compose start             # start them again
docker compose restart api       # restart one service
docker compose down              # remove containers and network; the db-data volume stays
```

Every service has restart: unless-stopped: Docker starts a service again when it exits, and starts the stack again when the Docker daemon starts, for example after a reboot. A service stopped by hand with docker compose stop or docker compose down stays stopped.

> **Warning:** docker compose down -v also deletes the db-data volume. Every order, quote, credit movement and outbox event is lost, and the next start seeds an empty database. Take a backup first.

Health and metrics

Endpoint	Answer	Use
GET http://localhost:8080/actuator/health	200 {"groups":["liveness","readiness"],"status":"UP"}	Overall health, UP when the database is reachable. No tenant headers
GET	200 {"status":"UP"}	Liveness probe

http://localhost:8080/actuator/health/liveness		
GET http://localhost:8080/actuator/health/readiness	200 {"status":"UP"}	Readiness probe

Only health is exposed over HTTP: GET /actuator lists only the health links, and /actuator/metrics answers 404. The API registers two gauges with Micrometer (OutboxConfig): outbox_pending, the delivery backlog, which grows while push or the ERP is down, and outbox_dead, the events that need an operator. No HTTP endpoint exposes them in this delivery, so read the same numbers in the database:

```
SELECT status, destination, count(*) FROM outbox_events GROUP BY status, destination ORDER BY 1, 2;
```

Health answers only on the API port. The client's origin forwards /api/ and nothing else, so http://localhost:8081/actuator/health returns the client page, not the health body.

Logs and correlation id

```
# From orderflow-engine-infra
docker compose logs -f api                  # follow the API
docker compose logs --tail 100 web          # last nginx lines
docker compose logs api | grep <correlationId>  # every line of one request
```

Every API log line carries the correlation id in brackets after the thread name: [<correlationId>] on a line written while serving a request, and [] on a line of a scheduled job. For example:

```
2026-10-17T14:31:12.114Z  INFO 1 --- [orderflow-engine-api] [   scheduling-1] [] c.o.e.adapters.out.erp.FakeErpGateway    : ERP order CO-000001 upserted from event 01a14a46-1ede-7b0b-9b91-bb601a4a11bd
```

CorrelationIdFilter reuses the request's X-Correlation-Id when it matches [A-Za-z0-9._-]{1,128}, and generates a UUID otherwise. It returns the id in the X-Correlation-Id response header and in the correlationId field of every error body. The web client shows its first 8 characters with each problem notice, so support can find the request in the logs. A 500 INTERNAL_ERROR is logged at ERROR level with its stack trace, and its body never exposes internals.

Logger	Level	Message	What to do
OutboxRelay	WARN	Outbox event ... failed attempt N of M, retrying at ...	Nothing yet: the relay retries with backoff
OutboxRelay	ERROR	Outbox event ... is DEAD for order ... after N attempts, needs an operator	Fix the cause, then requeue the event
OutboxRelay	WARN	Outbox event ... was	Nothing: another poll

		taken over after its lease expired, outcome not recorded	owns the event, and consumers deduplicate by event_id. Frequent lines suggest raising API_OUTBOX_LEASE
HousekeepingJob	INFO	Housekeeping purged N idempotency keys, N unconfirmed quotes and N sent outbox events	Nothing: the hourly purge ran
FakeErpGateway, FakePushGateway	INFO	ERP order CO-000001 upserted from event ..., Push notification ... shown to customer ... for order ...	Nothing: delivery reached the fake
ProblemDetailsAdvice	ERROR	Unexpected error, answered as INTERNAL_ERROR with status 500	A defect: search the lines of its correlation id

Outbox delivery

Each confirmed order writes one event for push and one for the ERP, in the order's own transaction. The relay delivers them later:

Status	Meaning	Next
PENDING	Due at next_attempt_at, or leased until locked_until while a poll sends it	The relay claims due events with FOR UPDATE SKIP LOCKED, so several API instances can run at once
SENT	The gateway accepted it	Purged by housekeeping after API_HOUSEKEEPING_RETENTION
DEAD	API_OUTBOX_MAX_ATTEMPTS failed attempts, or at once on an error that retrying cannot fix, such as an ERP 4xx	Never purged. It waits for an operator

- **Never lost:** if the API stops between sending and marking, the lease expires and the event is sent again.
- **Never applied twice:** consumers deduplicate by event_id. The fakes do so, a real push would use it as the collapse id, and the ERP would upsert by order number.
- **External system down:** the order stays confirmed and the effect waits in the outbox. Delivery resumes when the system recovers.

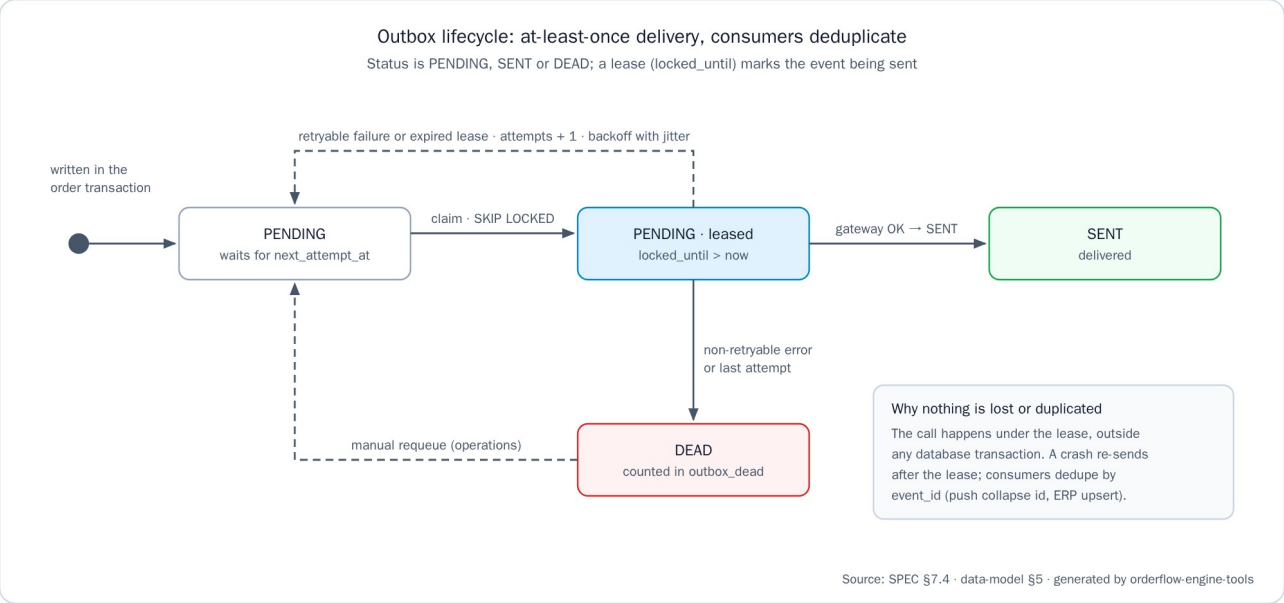


Figure 3. Outbox lifecycle: PENDING, leased, SENT, retry with backoff, and DEAD

To see what is not delivered yet, in psql:

```
SELECT tenant_id, id, order_id, destination, status, attempts, next_attempt_at,
last_error
FROM outbox_events
WHERE status <> 'SENT'
ORDER BY created_at DESC;
```

Requeue a DEAD event

Fix the cause first, as last_error records it. Then list the DEAD events and requeue one by its tenant_id and id, replacing co-alpha and <id> with the values the first query returned:

```
SELECT tenant_id, id, order_id, destination, attempts, last_error, created_at
FROM outbox_events
WHERE status = 'DEAD'
ORDER BY created_at DESC;

UPDATE outbox_events
SET status = 'PENDING', attempts = 0, next_attempt_at = now(), locked_until = NULL
WHERE tenant_id = 'co-alpha' AND id = '<id>' AND status = 'DEAD';
```

Why this is safe, from the operations section of orderflow-engine-api/README.md:

- The relay claims PENDING events whose next_attempt_at has passed, so the next poll sends it.
- attempts = 0 gives the event its full retry budget again, and last_error stays as evidence of the last failure.
- The status = 'DEAD' condition makes the statement repeatable: on an event already requeued it changes nothing.
- Consumers deduplicate by event_id, so requeuing an event that was in fact delivered has no second effect.

Source: orderflow-engine-docs/delivery/technical-manual.md · p. 13

Retention housekeeping

A housekeeping job runs every hour for every tenant and logs what it purged.

Data	Kept for	Key
Idempotency keys	24 hours. A later retry of a confirmed quote still gets QUOTE_ALREADY_CONFIRMED with the order number	API_IDEMPOTENCY_RETENTION
Unconfirmed quotes	7 days after they expire	API_HOUSEKEEPING_RETENTION
SENT outbox events	7 days after they were written	API_HOUSEKEEPING_RETENTION
Orders, their quotes, credit movements and DEAD events	Never deleted	—

Inspect the database

The database port is not published on the host. Open psql inside the db container, with DB_USER and DB_NAME if .env changed them:

```
# From orderflow-engine-infra
docker compose exec db psql -U orderflow -d orderflow
```

```
\dt
SELECT id, filename, orderexecuted FROM databasechangelog ORDER BY orderexecuted;
SELECT tenant_id, customer_id, credit_limit, consumed FROM credit_accounts ORDER BY 1, 2;
SELECT tenant_id, order_number, customer_id, confirmed_at FROM orders ORDER BY confirmed_at DESC LIMIT 10;
```

> **Warning:** quotes, quote lines and their discount components are insert-only, and an order stores no amount: its figures are read from its quote. Never update them by hand. Catalog and lever changes go through a changeset (see Maintenance).

Backup and restore

A backup is a pg_dump in PostgreSQL's custom format, streamed out of the db container:

```
# From orderflow-engine-infra, in bash
docker compose exec -T db pg_dump -U orderflow -d orderflow --format=custom > orderflow-backup.dump
```

To restore it, stop the API so nothing writes during the restore, restore over the existing objects, and start the API again:

```
# From orderflow-engine-infra, in bash
docker compose stop api
docker compose exec -T db pg_restore -U orderflow -d orderflow --clean --if-exists <
orderflow-backup.dump
docker compose start api
```

-T disables the pseudo-terminal, so the binary dump passes through the redirection intact. The dump includes databasechangelog, so after a restore Liquibase applies only the changesets newer than the backup. Restore into PostgreSQL 17, the version of the stack. On 2026-10-17 a dump of the running stack was restored into an empty PostgreSQL 17 and again over itself with --clean --if-exists, and its orders and its 33 changesets were intact.

The API

The full contract, with every request, response and error, is orderflow-engine-docs/spec/api-contract.md.

Endpoints

Id	Operation	Success	Notes
E-01	POST /api/v1/quotes	201 with Location: /api/v1/quotes/{quoteId}	Prices the cart and seals the quote. Body: paymentMethod (CASH or CREDIT) and lines of {sku, qty}
E-02	GET /api/v1/quotes/{quoteId}	200	The quote with its derived status: OPEN, CONFIRMED or EXPIRED. credit is recomputed while it is OPEN
E-03	POST /api/v1/orders	201 with Location: /api/v1/orders/{orderId}	Confirms a quote. Requires Idempotency-Key. Body: {"quoteId": "..."}
E-04	GET /api/v1/orders/{orderId}	200	The order, rendered from its quote
E-05	GET /api/v1/orders? customerId={id}&size=20	200	The customer's orders, newest first. size defaults to 20, and a value above 100 is read as 100
E-06	GET /actuator/health	200	Health. No tenant headers
E-07	GET /api/v1/products	200	The tenant's products in SKU order, with list

			prices before levers and taxes. It only reads

Order numbers are <COUNTRY>-NNNNNN, for example CO-000001, drawn from one sequence per country.

Headers

Header	Direction	Rule
X-Tenant-Id	Request	Required on every business operation. An unknown tenant answers 400 TENANT_NOT_FOUND
X-Customer-Id	Request	Required on every business operation, a UUID
Idempotency-Key	Request, E-03 only	Required, matching ^[A-Za-z0-9_-]{8,64}$. A UUID is recommended, one per quote
Accept-Language	Request	Optional: es (the default) or en. It picks the language of title and detail in errors
X-Correlation-Id	Request and response	Optional in the request. Echoed when it matches [A-Za-z0-9._-]{1,128}, otherwise replaced by a new UUID
Location	Response	On the 201 of E-01 and E-03
Idempotent-Replayed	Response	true when E-03 returns a stored response

> **Warning:** the API has no authentication, by the scope of the assignment: tenant and customer come from headers, and in production a gateway would authenticate the caller and inject them (contract gap GAP-01). Do not publish ports 8080 or 8081 on an untrusted network.

Idempotent confirmation

Situation	Response
Same key, same body, already completed	The stored response, byte for byte, with Idempotent-Replayed: true. No second order and no second credit consumption
Same key, different body	422 IDEMPOTENCY_KEY_REUSED
Same key while the first request is still running	Waits, then replays. 409 IDEMPOTENCY_KEY_IN_PROGRESS when the 2-second lock_timeout passes first
A failed request: validation, expired quote, credit exceeded	Not stored. The transaction rolls back and releases the key, so the client can fix the cause and retry
A new key on a confirmed quote	409 QUOTE_ALREADY_CONFIRMED, with orderNumber and orderId in the body

Error codes

Errors are application/problem+json (RFC 9457) with type, title, status, detail, instance, a stable code and the correlationId:

{"detail":"One of the products does not exist or is not sold by this tenant.","instance":"/api/v1/quotes","status":422,"title":"Unknown product","type":"https://orderflow-engine.example/problems/unknown-sku","code":"UNKNOWN_SKU","correlationId":"manual-check-001"}

Code	Status	Meaning
VALIDATION_FAILED	400	Malformed request: a missing header, a value that is not a UUID, a body that is not a JSON object, a missing or unknown field value
INVALID_QUANTITY	400	A quantity at or below 0, not an integer, or above 1,000,000, also after repeated SKUs are merged
CART_EMPTY	400	lines is empty
TENANT_NOT_FOUND	400	Unknown X-Tenant-Id
IDEMPOTENCY_KEY_MISSING	400	E-03 without Idempotency-Key
IDEMPOTENCY_KEY_INVALID	400	Idempotency-Key does not match its pattern
QUOTE_NOT_FOUND	404	Unknown quote, or one of another tenant or customer
ORDER_NOT_FOUND	404	Unknown order, or one of another tenant or customer
CUSTOMER_NOT_FOUND	404	Unknown customer for the tenant, or a history asked for another customerId
REQUEST_NOT_SUPPORTED	404, 405, 406 or 415	Unknown route, method, media type or accepted type, with the status the framework chose
QUOTE_EXPIRED	409	The quote passed its expiresAt
QUOTE_ALREADY_CONFIRMED	409	The quote was confirmed under another key. The body carries orderNumber and orderId
CREDIT_LIMIT_EXCEEDED	409	The total does not fit the available credit at confirmation. Nothing is written
IDEMPOTENCY_KEY_IN_PROGRESS	409	The same key is still being processed when lock_timeout passes
UNKNOWN_SKU	422	A SKU unknown, or not sold by this tenant
CREDIT_NOT_ENABLED	422	CREDIT asked by a customer without a credit account
IDEMPOTENCY_KEY_REUS	422	Same key, different request

ED		
INTERNAL_ERROR	500	An unexpected failure, logged with the correlation id

Foreign quotes, orders and histories answer 404, never 403, so the API never reveals that they exist. Source: ErrorCode.java and the contract's behavior decisions BD-01 to BD-04.

Walk-through with curl

This session quotes 25 colas on credit for Tienda Norte, confirms the quote, replays the confirmation and reads the order and the history. It was run against a running stack on 2026-10-17.

1. Set the session values and read the catalog:

```
API=http://localhost:8080
TENANT='X-Tenant-Id: co-alpha'
CUSTOMER='X-Customer-Id: 01a13cdb-cc00-7001-8001-000000000001'
curl -s "$API/api/v1/products" -H "$TENANT" -H "$CUSTOMER"
```

```
{"currency":"COP","items":[{"sku":"BEV-COLA-1500","description":"Gaseosa cola 1,5
l","listPrice":"5200","taxRate":"0.190000"},{"sku":"BEV-
COLA-350","description":"Gaseosa cola 350
ml","listPrice":"2500","taxRate":"0.190000"}, ...]}
```

2. Create the quote, keep its id and read it back:

```
quote_id=$(curl -s "$API/api/v1/quotes" -H 'Content-Type: application/json' -H
"$TENANT" -H "$CUSTOMER" \
  -d '{"paymentMethod":"CREDIT","lines":[{"sku":"BEV-COLA-350","qty":25}]}' \
  | sed -E 's/.*"quoteId":"([^"]+)".*/\1/')
curl -s "$API/api/v1/quotes/$quote_id" -H "$TENANT" -H "$CUSTOMER"
```

The breakdown is golden test 1: a gross of 62500, a tier discount of 2750 (TIER:01a13cdb-cc00-7001-8003-000000000001), a tax of 11353 and "total":"71103", with "status":"OPEN", an expiresAt 15 minutes ahead and credit.fits true. credit.available is 500000 on a fresh stack and goes down with each credit order.

3. Confirm it with a new idempotency key:

```
key=5d1f3a2b-8c4e-4f6a-9b7d-0e1f2a3b4c5d
curl -i "$API/api/v1/orders" -H 'Content-Type: application/json' -H "$TENANT" -H
"$CUSTOMER" \
  -H "Idempotency-Key: $key" -d "{\"quoteId\":\"$quote_id\"}"
```

```
HTTP/1.1 201
X-Correlation-Id: <uuid>
Location: /api/v1/orders/<orderId>
Content-Type: application/json

{"orderId":"<orderId>","orderNumber":"CO-000001","status":"CONFIRMED","quoteId":"<quoteId>","confirmedAt":"...","currency":"COP","lines":[...],"totals":{...,"total":"71103"},"paymentMethod":"CREDIT"}
```

CO-000001 is the first order of Colombia on a fresh stack. The totals are the quote's.

4. Send the same command again. The answer is the same body, byte for byte, with Idempotent-Replayed: true, and no second order is created. Keep the order id from it:

```
curl -i "$API/api/v1/orders" -H 'Content-Type: application/json' -H "$TENANT" -H "$CUSTOMER" \
  -H "Idempotency-Key: $key" -d "{\"quoteId\":\"$quote_id\"}"
order_id=$(curl -s "$API/api/v1/orders" -H 'Content-Type: application/json' -H "$TENANT" -H "$CUSTOMER" \
  -H "Idempotency-Key: $key" -d "{\"quoteId\":\"$quote_id\"}" | sed -E 's/.*"orderId":"([^"]+)".*/\1/')
```

5. Read the order and the history. The order comes first as soon as the 201 arrives:

```
curl -s "$API/api/v1/orders/$order_id" -H "$TENANT" -H "$CUSTOMER"
curl -s "$API/api/v1/orders?customerId=01a13cdb-cc00-7001-8001-000000000001" -H "$TENANT" -H "$CUSTOMER"
```

```
{"currency":"COP","items":[{"orderId":"<orderId>","orderNumber":"CO-000001","confirmedAt":"...","total":"71103"}]}
```

6. See an error in English, with a correlation id of your own:

```
curl -i "$API/api/v1/quotes" -H 'Content-Type: application/json' -H "$TENANT" -H "$CUSTOMER" \
  -H 'Accept-Language: en' -H 'X-Correlation-Id: manual-check-001' \
  -d '{"paymentMethod":"CASH","lines":[{"sku":"NOPE-1","qty":1}]}'
```

It answers 422 UNKNOWN_SKU with "correlationId":"manual-check-001", the body shown under Error codes.

> **Note:** to repeat the walk-through, use a new key. The same key with a new quote is a different request and answers 422 IDEMPOTENCY_KEY_REUSED for 24 hours. A new key on the quote already confirmed answers 409 QUOTE_ALREADY_CONFIRMED with its order number.

Tests and quality gates

The workspace command

```
# From orderflow-engine-infra
bash scripts/test-all.sh
```

```
# From orderflow-engine-infra, on Windows with PowerShell 7
pwsh -File scripts/test-all.ps1
```

test-all finds the workspace root two levels above the script and runs, in order:

Step	Command	Folder
1. API verify	./mvnw verify (mvnw.cmd verify in PowerShell)	orderflow-engine-api
2. Web tests	docker build --target test .	orderflow-engine-web
3. End-to-end	bash scripts/e2e.sh (scripts/e2e.ps1 in PowerShell)	orderflow-engine-infra
4. Documentation check	node orderflow-engine-tools/docs-check/check-docs.mjs	Workspace root
5. Integrity check	node orderflow-engine-tools/integrity/check-integrity.mjs	Workspace root

It stops at the first failure with test-all: FAIL (<step>, exit code <n>) and that exit code. When every step passes, it ends with test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check).

API build and tests

```
# From orderflow-engine-api
./mvnw verify
```

This single command builds the service and runs every test: unit tests, golden tests (tagged @Tag("golden"), fixtures in src/test/resources/golden/), contract tests (*ContractIT), architecture tests (ArchUnit) and integration tests against a real PostgreSQL 17 started by Testcontainers, including the Liquibase rollback test. It enforces the JaCoCo gate: at least 90% of the lines and branches of com.orderflow.engine.pricing and its subpackages. The coverage report is target/site/jacoco/index.html.

Mutation testing

```
# From orderflow-engine-api
./mvnw -Pmutation verify
```

The optional mutation profile runs PIT on com.orderflow.engine.pricing.domain and its subpackage lever, and fails below 80% of detected mutations. It is slower, so it runs before a delivery rather than on every build.

Web tests

```
# From orderflow-engine-web
docker build --target test .
```

The test stage of the web Dockerfile starts from the Playwright image of the pinned version (mcr.microsoft.com/playwright:v1.64.1-noble), runs npm ci, then npm test: the literal-text check (scripts/no-literal-text.mjs, no template holds user text outside the dictionaries), then every spec in headless Chromium, with the layout measured at 768 × 1024 and 375 × 812. A failing spec fails the build. The host needs only Docker.

To run the specs on the host instead, with a supported Node: npm ci, npx playwright install chromium once, then npm test.

End-to-end flow

```
# From orderflow-engine-infra
bash scripts/e2e.sh
```

1. e2e/check-playwright-version.mjs checks that the suite uses the Playwright version that orderflow-engine-web pins.
2. docker build makes the runner image from e2e/Dockerfile.
3. docker compose --project-name ofe-e2e up --build --detach --wait starts a full stack of its own, with the API on 18080 and the client on 18081.
4. The runner joins that stack's network and walks cart, quote, confirmation, order and My orders in a browser, once in Spanish and once in English, failing on any browser console error.
5. docker compose --project-name ofe-e2e down --volumes --remove-orphans always removes that stack and its database, also when a step fails, so every run starts from the seeds. After a failure it first prints the end of the stack's logs.

> **Note:** the end-to-end stack has its own project name and ports, so it never touches a stack started by hand on 8080 and 8081. To run the suite from the host against a running stack instead, from orderflow-engine-infra/e2e: npm ci, npx playwright install chromium, then npx playwright test with E2E_BASE_URL=http://localhost:8081.

Documentation and integrity checks

```
# From the workspace root
node orderflow-engine-tools/docs-check/check-docs.mjs
node orderflow-engine-tools/integrity/check-integrity.mjs
```

Check	What it verifies
check-docs	Markdown locations and index, language pairs with the same number of headings, links that never leave their repository, front matter, required headings, ADR length, identical i18n keys in every language, the progress chart and the sources of the Word and PDF deliverables
check-integrity	Required .gitignore, .dockerignore and .gitattributes

	entries, .env.example wherever a repository runs with Docker, Docker files that stay inside their repository, and no secrets

Verified evidence

From orderflow-engine-docs/handoff/HANDOFF.md §3:

Gate	Result	Threshold
API tests	333 unit + 308 integration tests green, the six golden tests among them	All green
JaCoCo on pricing.domain and pricing.domain.lever	100% lines and branches	90%
JaCoCo on the whole service	99.2% lines / 98.0% branches	—
PIT	185/185 mutations detected (100%)	80%
Web specs	209 specs in 15 files in headless Chromium	All green
End-to-end flow	2/2 (es and en)	All green
test-all	OK on fresh clones on 2026-10-17	—

Phases: preparation T-00, scaffold T-01, core T-02..T-16 + T-18, web T-17, T-19, T-20 — all closed on main.

Maintenance

Where code goes

The API is a hexagonal monolith. Every package lives under com.orderflow.engine:

Package	Kind	Put here
pricing.domain, pricing.domain.lever	Domain, pure	Money, the currency policy, the pricing engine, the lever steps and the breakdown. Only JDK imports
quoting	Application	Quote and product-list use cases and their ports (SnapshotProvider, QuoteRepository, CreditAdvisor, ProductCatalog)
ordering	Application	Confirmation, order and history use cases and their ports (OrderRepository, CreditLedger, IdempotencyStore, OutboxWriter)
outbox	Application	OutboxRelay, HousekeepingJob and their ports (PushNotificationPort, ErpPublisherPort, OutboxRepository)
adapters.in.web	Driving adapter	Controllers, DTOs, ProblemDetailsAdvice, ErrorCode, locale and correlation filters

adapters.out.persistence	Driven adapter	JdbcClient repositories with explicit SQL and locks
adapters.out.push, adapters.out.erp	Driven adapters	The fake gateways. A real client replaces one binding in config
config	Composition root	Wiring of ports to adapters and typed properties

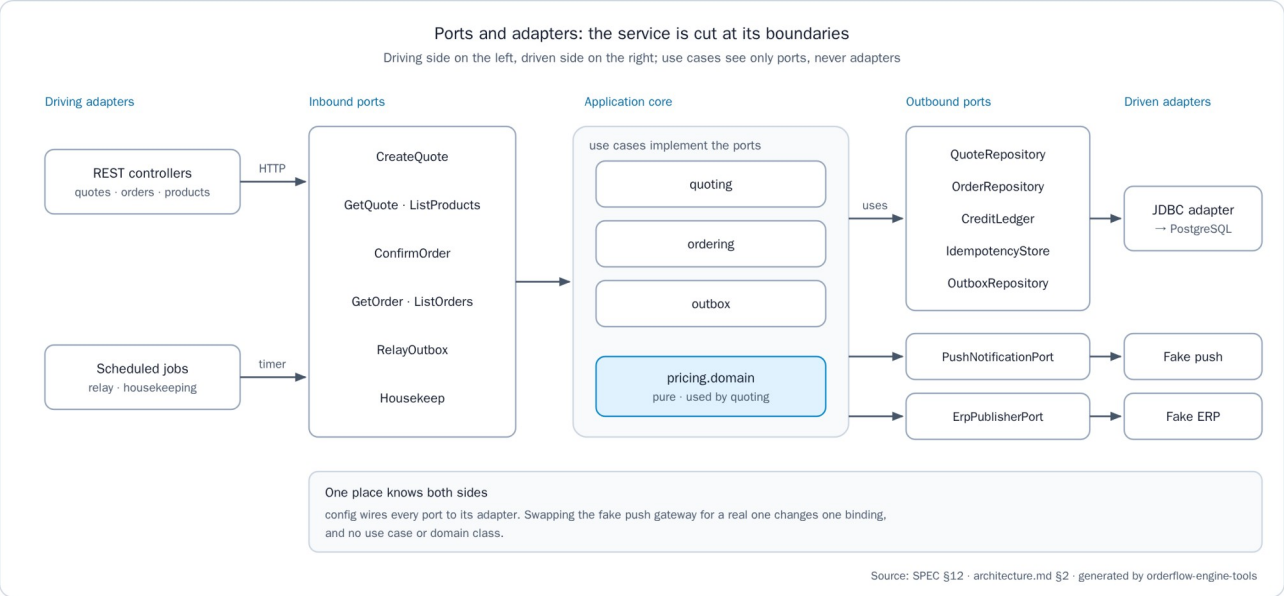


Figure 4. Ports and adapters: driving adapters, inbound ports, the application core, outbound ports and driven adapters

Dependencies point inward: adapters, then application, then domain. Two ArchUnit tests fail the build when the rule breaks:

- DomainPurityArchTest: the pricing domain depends only on java.. and on itself, never on java.sql, never on float or double, and never on java.util.Currency.
- TransactionBoundaryArchTest: a transaction starts only in an application use case; a transactional use case never reaches push, the ERP or an HTTP client; use cases depend on ports, never on adapters or wiring; and the confirmation never prices a cart again.

User text never lives in code. A new API error needs a value in ErrorCode and the keys error.<CODE>.title and error.<CODE>.detail in every bundle of src/main/resources/i18n/ (messages.properties, messages_es.properties, messages_en.properties). Client text goes to orderflow-engine-web/public/i18n/es.json and en.json, with the same keys in both.

Liquibase changesets

Liquibase owns the schema and the data that has no screen: stores, products, prices and levers. Each change is a new changelog file:

1. Create orderflow-engine-api/src/main/resources/db/changelog/<year>/<date>-<nnn>-<what>.yaml, for example 2026/2026-10-17-001-seed-mx.yaml.
2. Set logicalFilePath to its path under src/main/resources, so the file can move without being applied again.

3. Give every changeSet a unique id, author: orderflow-engine, and a rollback that removes exactly what it adds.
4. Include the file at the end of db/changelog/db.changelog-master.yaml, which lists every changelog in the order it is applied.
5. Never edit a changeset that was already applied. Correct it with a new one, as 2026-10-16-001-seed-names-in-spanish.yaml renamed the seeded rows.
6. Write every amount at the scale of the tenant's currency (0 decimals for COP, 2 for USD, GTQ and PEN). No database check can enforce it, so PriceScaleValidationIT does.
7. Run ./mvnw verify. LiquibaseRollbackIT rolls every changeset back to an empty schema and applies them again.
8. Rebuild and restart the API: docker compose up --build -d in orderflow-engine-infra. Liquibase applies the new changesets at start.

> **Note:** The integration tests pin the seeded demonstration data. A change whose changeset alters a seeded tenant updates, in the same change, ProductContractIT (the exact E-07 body of each tenant), LiquibaseRollbackIT (the tenants, countries and currencies, every product and customer name, and the order number sequences), PriceScaleValidationIT (the tenants with seeded prices) and the tests that assert amounts priced from the seeds, such as the lever cases of QuoteLeversIT and QuoteContractIT. Otherwise ./mvnw verify fails. LiquibaseRollbackIT first rolls back every changeset from the Guatemala and Peru changelog onward, so a row added by a later changelog goes only into the names it expects after an update (assertFourCountries), in their sort order, and not into the names it expects after that rollback.

```
databaseChangeLog:
 - logicalFilePath: db/changelog/2026/2026-10-17-001-<what>.yaml

 - changeSet:
     id: <unique-id>
     author: orderflow-engine
     changes:
       - sql:
           sql: |
             <sql>
     rollback:
       - sql:
           sql: |
             <rollback sql>
```

```
 - include:
     file: 2026/2026-10-17-001-<what>.yaml
     relativeToChangelogFile: true
```

The seeds use fixed UUIDv7 ids with a readable pattern, 01a13cdb-cc00-70TT-80KK-NNNNNNNNNNNN: TT is the tenant (01 co-alpha, 02 ec-alpha, 03 gt-alpha, 04 pe-alpha), KK is the kind (01 customer, 02 product, 03 tier rule, 04 combo, 05 gift) and NNNNNNNNNNNN is the sequence. New seed rows keep that pattern.

Change a price or deactivate a lever

A price change applies to new quotes only. An open quote keeps the price it was quoted with until it expires, and a confirmed order never changes, which QuoteConfirmRegressionIT proves. A lever is deactivated, never deleted, because quote lines keep pointing at the rule that produced them.

```
 - changeSet:
     id: co-alpha-cola-350-price-2600
     author: orderflow-engine
     changes:
       - sql:
           sql: UPDATE products SET list_price = 2600 WHERE tenant_id = 'co-alpha'
AND sku = 'BEV-COLA-350'
     rollback:
       - sql:
           sql: UPDATE products SET list_price = 2500 WHERE tenant_id = 'co-alpha'
AND sku = 'BEV-COLA-350'

 - changeSet:
     id: co-alpha-deactivate-snacks-gift
     author: orderflow-engine
     changes:
       - sql:
           sql: UPDATE gift_rules SET active = false WHERE tenant_id = 'co-alpha' AND
id = '01a13cdb-cc00-7001-8005-000000000001'
     rollback:
       - sql:
           sql: UPDATE gift_rules SET active = true WHERE tenant_id = 'co-alpha' AND id
= '01a13cdb-cc00-7001-8005-000000000001'
```

Both examples change seeded co-alpha data that the tests pin: ProductContractIT expects the list price 2500, and QuoteLeversIT, QuoteContractIT and QuoteConfirmRegressionIT assert quotes priced from it, with the gift GIFT:01a13cdb-cc00-7001-8005-000000000001 and totals such as 71103. Update those expectations in the same change.

Add a product

1. In a changeset, insert the product. Its family must exist for the tenant (insert it first when it is new), its SKU is unique per tenant, its tax rate is between 0 and 1, and its price is tax-exclusive at the currency's scale.
2. Optionally add its image as orderflow-engine-web/public/products/<SKU>.svg. A product without one shows the placeholder.

3. Add the product, in SKU order, to the E-07 body that ProductContractIT pins for its tenant (CO_PRODUCTS for co-alpha), and to the product names that LiquibaseRollbackIT expects after an update, then run ./mvnw verify.
4. Rebuild the API, and the web image if an image was added. E-07 and the cart list the product at once, and E-01 prices it with the tenant's levers.

```
 - changeSet:
     id: seed-co-alpha-juice
     author: orderflow-engine
     changes:
       - sql:
           sql: |
             INSERT INTO families (tenant_id, code) VALUES ('co-alpha', 'juices');
             INSERT INTO products (tenant_id, id, sku, name, family_code, tax_rate,
list_price) VALUES
               ('co-alpha', '01a13cdb-cc00-7001-8002-000000000005', 'BEV-JUICE-300',
'Jugo de naranja 300 ml', 'juices', 0.190000, 3200)
     rollback:
       - sql:
           sql: |
             DELETE FROM products WHERE tenant_id = 'co-alpha' AND sku = 'BEV-
JUICE-300';
             DELETE FROM families WHERE tenant_id = 'co-alpha' AND code = 'juices'
```

Product names are tenant data in the tenant's language, shown as stored. The ES/EN switch translates the interface, not the catalog.

Add a lever

Lever	Tables	Rules the schema enforces
Tier discount	tier_rules (product_id or family_code, active) and tier_brackets (from_qty, percent)	Exactly one target. At most one active rule per SKU and per family, so replacing a rule deactivates the old one in the same changeset. from_qty of 1 or more, percent between 0 and 1. Brackets are marginal, and a SKU rule takes precedence over a family rule
Combo	combo_rules (combo_price, active) and combo_items (product_id, min_qty)	min_qty of 1 or more. Combos are evaluated in id order, and a combo that would not lower the price consumes nothing
Gift	gift_rules (trigger, gift_product_id, gift_qty, repeatable, active)	Exactly one trigger: trigger_product_id with min_qty, trigger_family_code with min_qty, or min_gross alone. gift_qty of 1 or more

A tier rule on a family, with its brackets, as a changeset:

```
 - changeSet:
     id: seed-co-alpha-waters-tier
     author: orderflow-engine
     changes:
       - sql:
           sql: |
             INSERT INTO tier_rules (tenant_id, id, product_id, family_code, active)
VALUES
               ('co-alpha', '01a13cdb-cc00-7001-8003-000000000002', NULL, 'waters', true);
             INSERT INTO tier_brackets (tenant_id, tier_rule_id, from_qty, percent)
VALUES
               ('co-alpha', '01a13cdb-cc00-7001-8003-000000000002',  1, 0.000000),
               ('co-alpha', '01a13cdb-cc00-7001-8003-000000000002', 24, 0.050000)
     rollback:
       - sql:
           sql: |
             DELETE FROM tier_brackets WHERE tenant_id = 'co-alpha' AND tier_rule_id
= '01a13cdb-cc00-7001-8003-000000000002';
             DELETE FROM tier_rules WHERE tenant_id = 'co-alpha' AND id = '01a13cdb-
cc00-7001-8003-000000000002'
```

The combo and gift shapes are in the country example below. The order in which levers apply is fixed by the engine, not by data: gifts, combos, tiers, net, taxable base, tax, total and credit (orderflow-engine-api/README.md, "Lever order").

Add a store

A store of the web client is a customer of a tenant.

1. In a changeset, insert the customer and, if it buys on credit, its credit account (consumed starts at 0 and can never exceed credit_limit).
2. Add the store to orderflow-engine-web/public/config/stores.json, next to the stores of its tenant. name is what the store picker shows.
3. Update the list that src/app/core/api-headers.interceptor.spec.ts pins in orderflow-engine-web, then run docker build --target test . there.
4. Add the customer, in tenant and id order, to the customer names that LiquibaseRollbackIT expects after an update, then run ./mvnw verify in orderflow-engine-api.
5. Rebuild both images: docker compose up --build -d in orderflow-engine-infra.

```
 - changeSet:
     id: seed-co-alpha-tienda-centro
     author: orderflow-engine
     changes:
       - sql:
           sql: |
             INSERT INTO customers (tenant_id, id, name) VALUES
               ('co-alpha', '01a13cdb-cc00-7001-8001-000000000003', 'Tienda Centro');
             INSERT INTO credit_accounts (tenant_id, customer_id, credit_limit,
consumed) VALUES
               ('co-alpha', '01a13cdb-cc00-7001-8001-000000000003', 300000, 0)
     rollback:
       - sql:
           sql: |
             DELETE FROM credit_accounts WHERE tenant_id = 'co-alpha' AND
customer_id = '01a13cdb-cc00-7001-8001-000000000003';
             DELETE FROM customers WHERE tenant_id = 'co-alpha' AND id =
'01a13cdb-cc00-7001-8001-000000000003'
```

```
 {
   "tenantId": "co-alpha",
   "customerId": "01a13cdb-cc00-7001-8001-000000000003",
   "name": "Tienda Centro"
 }
```

> **Note:** stores.json is part of the web image, so a change needs a rebuild of web. A store whose tenant or customer the database does not know answers 400 TENANT_NOT_FOUND or 404 CUSTOMER_NOT_FOUND in the client.

Add a country

A new country is data only. Guatemala and Peru were added this way (task T-20, 2026-10-16-002-seed-gt-and-pe.yaml) with no change to the engine or the API code, which is itself the proof that no currency, scale, rate or rule is hard-coded. Follow the same order of changesets:

1. The currency, when it is new, with its scale (currencies).
2. The country: currency, rounding mode, gift tax policy (EXEMPT or TAXED) and quote time to live (countries).
3. The order number sequence of the country, named order_number_seq_ followed by the country code in lower case. The order repository draws the next number from it and formats it as <COUNTRY>-NNNNNN, so the first order of Mexico is MX-000001.
4. The tenant: its id, country and brand (tenants; one tenant per country and brand).
5. The tenant's data: families, products, levers, customers and credit accounts.

The example below adds a tenant mx-alpha for Mexico. It is an example to adapt, not a file of the repository: the currency, rates, prices and the gift tax policy must come from the

country's real fiscal rules. Its SQL, rollback included, was run on 2026-10-17 against a copy of the stack's database, and the new sequence gave MX its number 1.

```
databaseChangeLog:
  - logicalFilePath: db/changelog/2026/2026-10-17-001-seed-mx.yaml

  - changeSet:
      id: seed-mx-currency
      author: orderflow-engine
      changes:
        - sql:
            sql: INSERT INTO currencies (code, scale) VALUES ('MXN', 2)
      rollback:
        - sql:
            sql: DELETE FROM currencies WHERE code = 'MXN'

  - changeSet:
      id: seed-mx-country
      author: orderflow-engine
      changes:
        - sql:
            sql: |
              INSERT INTO countries (code, currency_code, rounding_mode,
gift_tax_policy, quote_ttl) VALUES
                ('MX', 'MXN', 'HALF_UP', 'EXEMPT', '15 minutes')
      rollback:
        - sql:
            sql: DELETE FROM countries WHERE code = 'MX'

  - changeSet:
      id: create-mx-order-number-sequence
      author: orderflow-engine
      changes:
        - createSequence: {sequenceName: order_number_seq_mx, startValue: 1,
incrementBy: 1}
      rollback:
        - dropSequence: {sequenceName: order_number_seq_mx}

  - changeSet:
      id: seed-mx-tenant
      author: orderflow-engine
      changes:
        - sql:
            sql: INSERT INTO tenants (id, country_code, brand) VALUES ('mx-alpha', 'MX',
'alpha')
      rollback:
        - sql:
            sql: DELETE FROM tenants WHERE id = 'mx-alpha'
```

```
  - changeSet:
      id: seed-mx-alpha
      author: orderflow-engine
      changes:
        - sql:
            sql: |
              INSERT INTO families (tenant_id, code) VALUES
                ('mx-alpha', 'colas'),
                ('mx-alpha', 'waters');

              INSERT INTO products (tenant_id, id, sku, name, family_code, tax_rate,
list_price) VALUES
                ('mx-alpha', '01a13cdb-cc00-7005-8002-000000000001', 'BEV-COLA-350',
'Refresco de cola 350 ml', 'colas',  0.160000, 12.50),
                ('mx-alpha', '01a13cdb-cc00-7005-8002-000000000002', 'BEV-WATER-600',
'Agua 600 ml',             'waters', 0.160000,  9.00);

              INSERT INTO tier_rules (tenant_id, id, product_id, family_code, active)
VALUES
                ('mx-alpha', '01a13cdb-cc00-7005-8003-000000000001', '01a13cdb-cc00-
7005-8002-000000000001', NULL, true);

              INSERT INTO tier_brackets (tenant_id, tier_rule_id, from_qty, percent)
VALUES
                ('mx-alpha', '01a13cdb-cc00-7005-8003-000000000001',  1, 0.000000),
                ('mx-alpha', '01a13cdb-cc00-7005-8003-000000000001', 12, 0.050000);

              INSERT INTO gift_rules
                (tenant_id, id, trigger_product_id, trigger_family_code, min_qty, min_gross,
                 gift_product_id, gift_qty, repeatable, active) VALUES
                ('mx-alpha', '01a13cdb-cc00-7005-8005-000000000001', '01a13cdb-cc00-
7005-8002-000000000001', NULL, 24, NULL,
                 '01a13cdb-cc00-7005-8002-000000000002', 1, true, true);

              INSERT INTO customers (tenant_id, id, name) VALUES
                ('mx-alpha', '01a13cdb-cc00-7005-8001-000000000001', 'Tienda Monterrey'),
                ('mx-alpha', '01a13cdb-cc00-7005-8001-000000000002', 'Tienda Puebla');

              INSERT INTO credit_accounts (tenant_id, customer_id, credit_limit,
consumed) VALUES
                ('mx-alpha', '01a13cdb-cc00-7005-8001-000000000001', 10000.00, 0)
      rollback:
        - sql:
            sql: |
```

```
DELETE FROM credit_accounts WHERE tenant_id = 'mx-alpha';
DELETE FROM customers WHERE tenant_id = 'mx-alpha';
DELETE FROM gift_rules WHERE tenant_id = 'mx-alpha';
DELETE FROM tier_brackets WHERE tenant_id = 'mx-alpha';
DELETE FROM tier_rules WHERE tenant_id = 'mx-alpha';
DELETE FROM products WHERE tenant_id = 'mx-alpha';
DELETE FROM families WHERE tenant_id = 'mx-alpha'
```

Then finish the change:

1. Include the file at the end of db.changelog-master.yaml.
2. Add the new stores to stores.json and to the web spec that pins it (see Add a store).
3. Extend the tests that pin the seeded data: PriceScaleValidationIT (the tenants with seeded prices) and, in LiquibaseRollbackIT, ALL_TENANTS, the countries and currencies of assertFourCountries, the product and customer names it expects after an update, and the order number sequences that its first rollback removes (add S order_number_seq_mx beside GT_AND_PE_SEQUENCES).
4. Run ./mvnw verify and docker build --target test . in orderflow-engine-web, then rebuild the stack with docker compose up --build -d.
5. Quote and confirm once with X-Tenant-Id: mx-alpha: the order number is MX-000001.

Why there is no catalog screen

Catalog and lever administration is out of scope by design (orderflow-engine-docs/spec/SPEC.md §1), in the API and in the client. The assignment asks for a pricing engine and an idempotent order confirmation, not for catalog administration, so the migrations seed the data (orderflow-engine-api/README.md, "Not included"). A screen would also need the authentication that the scope leaves out. Changesets already give what such a screen would have to rebuild: every change is versioned and reviewed in a pull request, has a rollback that LiquibaseRollbackIT tests, and its amounts are checked against the currency scale by PriceScaleValidationIT. The only catalog operation is the read-only product list E-07, which feeds the cart. The client's store picker stands in for authentication with the seeded ids of stores.json; in production the identity would come from authentication, not from a list (contract gap GAP-04).

Troubleshooting

Symptom	Cause	Remedy
docker compose up fails with "port is already allocated" on 8080 or 8081	The API or web stack started alone, or another program, publishes the same port	Stop the other stack, or set another API_PORT or WEB_PORT in .env
The web container exits at start or restarts in a loop, and its log says the upstream host is not found	nginx resolves the proxy_pass host when it starts and exits if the name does not resolve (HANDOFF TR-12)	In the full stack, start api first, then docker compose up -d web. For the client alone, start the API on the host first. On Linux, extra_hosts maps

		host.docker.internal to the host gateway
/api/ answers 502 Bad Gateway after the API was rebuilt	nginx kept the address it resolved when it started	docker compose restart web
http://localhost:8081/actuator/health returns the client page	Only /api/ is forwarded by the client's origin	Use http://localhost:8080/actuator/health
The API restarts in a loop with a password authentication error	DB_PASSWORD changed after the db-data volume was created	Restore the previous value, or change the role's password in psql with ALTER ROLE. Deleting the volume also works, and deletes all data
The API does not start and Liquibase reports a validation or checksum error	A changeset that was already applied was edited	Revert the edit and make the change in a new changeset
./mvnw verify fails with ContainerLaunchException, not with an assertion	Under heavy parallel load, Testcontainers timed out waiting for PostgreSQL (HANDOFF TR-14)	Rerun on a quieter host before reading it as a defect
./mvnw or scripts/test-all.sh answers "Permission denied" on Linux or macOS	A clone made on Windows with core.filemode=false committed it without the executable bit (HANDOFF TR-07)	Run it as bash scripts/test-all.sh or sh mvnw verify. When committing, stage it with git add --chmod=+x and check that git ls-files -s shows 100755
test-all says OK, but the web specs did not run again	Docker's build cache served the test stage of an earlier build (HANDOFF TR-15)	docker build --no-cache-filter test --target test . in orderflow-engine-web
npm test or npm start on the host fails on the Node engine	Angular 22 needs Node 22.22.3 or later in the 22 line, or 24.15 or later (HANDOFF TR-13)	Run the specs in their container with docker build --target test ., or upgrade Node
The client shows TENANT_NOT_FOUND or CUSTOMER_NOT_FOUND for a store	stores.json names a tenant or customer that the database does not have	Align stores.json with the seeds, then rebuild web
A new product, store or price does not appear	The changelog is not included in the master changelog, or the API image was not rebuilt	Include it, then docker compose up --build -d
E-03 answers 409 IDEMPOTENCY_KEY_IN_P	The first request with that key was still running when	Retry with the same key and body: it replays the stored

ROGRESS	the 2-second lock_timeout passed	response once the first request ends
outbox_events has DEAD rows, or PENDING rows keep growing	Push or the ERP fails or is down	Read last_error and the relay's WARN and ERROR lines, fix the cause, then requeue the DEAD events

Quick command reference

```
# Stack, from orderflow-engine-infra
docker compose up --build -d
docker compose ps
docker compose logs -f api
docker compose restart web
docker compose stop
docker compose down                    # keeps the data; down -v deletes it
curl http://localhost:8080/actuator/health

# Database, from orderflow-engine-infra
docker compose exec db psql -U orderflow -d orderflow
docker compose exec -T db pg_dump -U orderflow -d orderflow --format=custom >
orderflow-backup.dump
docker compose exec -T db pg_restore -U orderflow -d orderflow --clean --if-exists <
orderflow-backup.dump

# Tests, from the workspace root
bash orderflow-engine-infra/scripts/test-all.sh                # every check, in order
(cd orderflow-engine-api && ./mvnw verify)                     # API, with the coverage
gate
(cd orderflow-engine-api && ./mvnw -Pmutation verify)          # PIT mutation testing
(cd orderflow-engine-web && docker build --target test .)      # web specs in a
container
(cd orderflow-engine-infra && bash scripts/e2e.sh)             # end-to-end flow
node orderflow-engine-tools/docs-check/check-docs.mjs          # documentation
check
node orderflow-engine-tools/integrity/check-integrity.mjs      # integrity check

# Standalone stacks, from the workspace root
(cd orderflow-engine-api && docker compose up --build)         # API and its
PostgreSQL
(cd orderflow-engine-web && docker compose up --build)         # client, /api/ to an
API on the host
```